

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP HCM
VIỆN CÔNG NGHỆ THÔNG TIN VÀ ĐIỆN, ĐIỆN TỬ



BÁO CÁO MÔN HỌC
CẤU TRÚC RỜI RẠC

TÊN ĐỀ TÀI

Xây dựng và chạy demo ứng dụng cho chức năng chính

Giảng viên: **Tăng Lê Ngọc Gia Huy**

Khóa học: **Cấu trúc rời rạc**

Mã lớp: **012012204402**

Nhóm: **Group_12**

Sinh viên nhóm thực hiện:

1. Hán Văn Trường
2. Lê Đặng Quỳnh Như
3. Lê Đào Mỹ Duyên
4. Dương Thị Thùy Trang
5. Nguyễn Lê Phương Uyên

MỤC LỤC

MỤC LỤC.....	2
1. Giới thiệu đề tài.....	7
Danh sách chức năng đã cài đặt.....	7
2. Công cụ, ngôn ngữ và quy ước cài đặt.....	8
3. Nhập đồ thị, vẽ và lưu hình.....	9
Đồ thị mẫu (samples/basic_undirected.txt) — 10 đỉnh, 14 cạnh.....	9
4. Các phương pháp biểu diễn đồ thị.....	10
Ma trận kề (Adjacency Matrix).....	10
Danh sách kề (Adjacency List).....	10
Danh sách cạnh (Edge List).....	10
5. Duyệt đồ thị BFS & DFS.....	11
Đối chiếu với kết quả chạy tay.....	12
Diễn giải chạy tay BFS(A).....	12
Diễn giải chạy tay DFS(A).....	13
6. Kiểm tra đồ thị hai phía (Bipartite).....	14
Trường hợp 1: Đồ thị hai phía (8 đỉnh, 12 cạnh).....	14
Trường hợp 2: Đồ thị không hai phía (8 đỉnh, 10 cạnh).....	14
7. Đường đi ngắn nhất: Dijkstra & Bellman–Ford.....	16
7.1 Dijkstra (9 đỉnh, 16 cạnh, trọng số không âm).....	16
7.2 Bellman–Ford (8 đỉnh, 14 cạnh, có cạnh âm, không có chu trình âm).....	17
8. Thuật toán Fleury và Hierholzer.....	19
Trường hợp có chu trình Euler (2 tam giác chung đỉnh A).....	19
Trường hợp có đường đi Euler (không phải chu trình).....	19
9. Thuật toán Prim và Kruskal.....	21
10. Ứng dụng thực tế: Tối ưu lộ trình xe quét rác.....	24
10.1 Mô tả bài toán.....	24
10.2 Mô hình hoá thành bài toán đồ thị.....	24
10.3 Vấn đề mấu chốt: mạng lưới thực tế không thỏa điều kiện Euler.....	24
10.4 Thuật toán xử lý: Bài toán Dijkstra, Ghép cặp trọng số nhỏ nhất và Hierholzer... 25	
10.5 Kết quả chạy chương trình.....	26
11. Kết luận.....	29
12. Hướng dẫn chạy chương trình.....	29

Yêu cầu môi trường.....	29
Chạy demo tổng hợp toàn bộ 8 phần.....	29
Chạy riêng bài toán Chinese Postman (Phần 8).....	29
Thử với đồ thị khác.....	29
13. Tài liệu tham khảo.....	30

LỜI CẢM ƠN

Lời đầu tiên, nhóm chúng em xin gửi lời cảm ơn chân thành đến Thầy Tăng Lê Ngọc Gia Huy giảng viên phụ trách học phần Cấu trúc rời rạc, đã tận tình giảng dạy, truyền đạt những kiến thức nền tảng quý báu về lý thuyết đồ thị trong suốt quá trình học tập. Những bài giảng của Thầy/Cô không chỉ giúp chúng em nắm vững lý thuyết mà còn khơi gợi niềm hứng thú để nhóm chủ động tìm hiểu sâu hơn, tự tay cài đặt và kiểm chứng từng thuật toán trong bài tập lớn này.

Chúng em cũng xin cảm ơn Thầy/Cô đã tạo điều kiện, đưa ra những yêu cầu và gợi ý mang tính định hướng trong suốt quá trình thực hiện đề tài, giúp nhóm từng bước hoàn thiện sản phẩm từ những phần cơ bản nhất đến các thuật toán nâng cao.

Trong quá trình thực hiện, mặc dù nhóm đã cố gắng nghiên cứu, tự cài đặt và kiểm chứng cẩn thận từng thuật toán, nhưng do kiến thức và kinh nghiệm thực tế còn hạn chế nên báo cáo khó tránh khỏi những thiếu sót. Nhóm chúng em rất mong nhận được sự góp ý, nhận xét từ Thầy/Cô để bài làm được hoàn thiện hơn, đồng thời đây cũng là những kinh nghiệm quý giá cho quá trình học tập và nghiên cứu sau này.

Chúng em xin chân thành cảm ơn.

LỜI MỞ ĐẦU

Trong thời đại công nghệ số phát triển mạnh mẽ như hiện nay, dữ liệu không còn tồn tại dưới dạng rời rạc, đơn lẻ mà thường mang tính liên kết chặt chẽ với nhau - từ mạng lưới giao thông đô thị, mạng xã hội, mạng máy tính, cho đến chuỗi cung ứng hàng hóa hay hệ thống định tuyến gói tin trên Internet. Để mô hình hóa và xử lý hiệu quả các mối quan hệ liên kết phức tạp đó, Lý thuyết đồ thị (Graph Theory) - một nhánh quan trọng của Toán học rời rạc - đã trở thành công cụ nền tảng không thể thiếu trong Khoa học máy tính hiện đại.

Học phần Cấu trúc rời rạc đã trang bị cho chúng em những kiến thức cốt lõi về đồ thị: từ các khái niệm cơ bản như đỉnh, cạnh, bậc của đỉnh, các cách biểu diễn đồ thị, cho đến những thuật toán kinh điển như duyệt đồ thị (BFS, DFS), tìm đường đi ngắn nhất (Dijkstra, Bellman-Ford), tìm chu trình/đường đi Euler (Fleury, Hierholzer), hay tìm cây khung nhỏ nhất (Prim, Kruskal). Đây đều là những thuật toán có tuổi đời hàng chục, thậm chí hàng trăm năm, nhưng vẫn giữ nguyên giá trị ứng dụng to lớn trong các hệ thống công nghệ hiện đại ngày nay.

Nhằm củng cố kiến thức lý thuyết đã học và rèn luyện kỹ năng lập trình, tư duy thuật toán, nhóm chúng em thực hiện đề tài "Xây dựng chương trình cài đặt và trực quan hóa các thuật toán trên đồ thị". Điểm đặc biệt của đề tài là toàn bộ thuật toán đều được nhóm tự tay cài đặt từ đầu, không sử dụng bất kỳ thư viện đồ thị dựng sẵn nào (như networkx, igraph...) cho phần thuật toán - các thư viện này chỉ được phép sử dụng ở khâu cuối cùng là trực quan hóa kết quả. Cách tiếp cận này buộc nhóm phải hiểu thấu đáo bản chất, độ phức tạp và các trường hợp biên của từng thuật toán, thay vì chỉ đơn thuần gọi hàm có sẵn.

Báo cáo được trình bày thành các phần chính: giới thiệu tổng quan, cấu trúc chương trình, phần cơ bản (biểu diễn đồ thị, duyệt đồ thị, kiểm tra tính hai phía, tìm đường đi ngắn nhất) và phần nâng cao (đồ thị Euler, cây khung nhỏ nhất). Mỗi thuật toán đều được trình bày kèm ví dụ minh họa, đối chiếu kết quả chương trình với kết quả tính tay, cũng như hình ảnh trực quan hóa để người đọc dễ dàng theo dõi và kiểm chứng.

Do thời gian thực hiện có hạn và đây là lần đầu nhóm tiếp cận việc tự cài đặt một hệ thống thuật toán đồ thị hoàn chỉnh, báo cáo chắc chắn không tránh khỏi những thiếu sót nhất định. Nhóm rất mong nhận được sự góp ý từ Thầy/Cô để hoàn thiện hơn.

Xin trân trọng cảm ơn và kính mời Thầy/Cô cùng các bạn theo dõi nội dung báo cáo.

DANH SÁCH THÀNH VIÊN VÀ PHÂN CÔNG NHIỆM VỤ

Nhóm gồm 5 thành viên, phân công công việc như bảng dưới đây:

STT	Họ và tên	Vai trò	Nhiệm vụ được phân công
1	Hán Văn Trường	Nhóm trưởng (Leader)	Điều phối chung, phân công và giám sát tiến độ; thiết kế lược đồ tổng thể; tổng hợp, định dạng toàn bộ báo cáo; kiểm tra chéo lần cuối; Đảm nhận phần Module trực quan hóa (visualize.py), tổng hợp main.py, đóng gói source code, viết báo cáo
2	Nguyễn Lê Phương Uyên	Thành viên	Module graph.py (cấu trúc đồ thị lỗi, 3 cách biểu diễn), BFS, DFS, kiểm tra Bipartite
3	Lê Đào Mỹ Duyên	Thành viên	Dijkstra, Bellman-Ford (shortest_path.py)
4	Dương Thị Thùy Trang	Thành viên	Prim, Kruskal, Union-Find (mst.py)
5	Lê Đặng Quỳnh Như	Thành viên	Euler: kiểm tra điều kiện, Fleury, Hierholzer (euler.py)

Ngoài phần việc được phân công riêng, mỗi thành viên đều tham gia họp nhóm định kỳ để trao đổi tiến độ, cùng nhau rà soát chéo các phần của người khác, và thống nhất quy ước chung khi viết nhằm đảm bảo toàn bộ báo cáo có phong cách nhất quán chung.

1. Giới thiệu đề tài

Đồ thị (Graph) là một trong những cấu trúc dữ liệu quan trọng nhất của Toán rời rạc, được ứng dụng rộng rãi trong khoa học máy tính: mạng máy tính, định tuyến GPS, mạng xã hội, lập lịch, thiết kế mạch điện, tối ưu vận tải – logistics, v.v. Bài tập nhóm này yêu cầu xây dựng một ứng dụng minh họa đầy đủ các thao tác cơ bản và nâng cao trên đồ thị, tự cài đặt thuật toán (không dùng thư viện đồ thị có sẵn), chỉ sử dụng thư viện bên ngoài cho phần trực quan hoá kết quả.

Trong bài tập lớn này, nhóm xây dựng một chương trình Python hoàn chỉnh, tự cài đặt toàn bộ các thuật toán đồ thị đã học trong môn Cấu trúc rời rạc (Không sử dụng các hàm thuật toán có sẵn của bất kỳ thư viện đồ thị nào như `networkx`, `igraph`...). Thư viện `matplotlib` và `networkx` được sử dụng ở trực quan hóa (vẽ hình, bố trí layout), trong phạm vi cho phép của đề bài. nhóm đã xây dựng lại toàn bộ về bộ dữ liệu mẫu với quy mô 8–10 đỉnh, 10–16 cạnh cho từng thuật toán, đảm bảo kết quả không thể suy ra bằng trực giác mà phải thực sự chạy thuật toán mới ra được.

Chương trình được tổ chức thành các module riêng biệt, rõ ràng, dễ kiểm thử và mở rộng, bao gồm: biểu diễn đồ thị, duyệt đồ thị, kiểm tra tính hai phía, tìm đường đi ngắn nhất, tìm chu trình/đường đi Euler, và tìm cây khung nhỏ nhất.

Danh sách chức năng đã cài đặt

- 1 Nhập đồ thị từ file / nhập tay; vẽ và lưu hình đồ thị (Phần 1).
- 2 Chuyển đổi qua lại giữa 3 cách biểu diễn: ma trận kề, danh sách kề, danh sách cạnh (Phần 2).
- 3 Duyệt đồ thị bằng BFS và DFS từ một đỉnh bất kỳ, so sánh với kết quả chạy tay (Phần 3).
- 4 Kiểm tra đồ thị hai phía – Bipartite (Phần 4).
- 5 Tìm đường đi ngắn nhất giữa 2 đỉnh bằng Dijkstra và Bellman–Ford, so sánh với kết quả chạy tay ở Phần 5.
- 6 Thuật toán Fleury và Hierholzer tìm đường đi/chu trình Euler.
- 7 Thuật toán Prim và Kruskal tìm cây khung nhỏ nhất – MST.
- 8 Ứng dụng Euler mở rộng tạo thêm một lớp hàm điều phối (`route_orchestrator.py`) cho "Thuật toán xử lý: Bài toán Dijkstra, Ghép cặp trọng số nhỏ nhất và Hierholzer" vào bài toán thực tế tối ưu lộ trình xe quét rác, xử lý cả trường hợp đồ thị không thỏa điều kiện Euler ở Phần 8.

2. Công cụ, ngôn ngữ và quy ước cài đặt

Ngôn ngữ lập trình: Python 3.

Thư viện sử dụng: matplotlib, networkx – dùng cho phần trực quan hoá (vẽ hình, tính tọa độ bố trí đỉnh).
Toàn bộ thuật toán (BFS, DFS, Dijkstra, Bellman-Ford, Prim, Kruskal, Fleury, Hierholzer, kiểm tra Bipartite, Union-Find, ghép cặp trong hàm điều phối...) đều được nhóm tự cài đặt từ đầu bằng cấu trúc dữ liệu Python thuần (list, dict, set).

Cấu trúc thư mục mã nguồn nộp kèm:

012012204402_Group_12/	
— src/	
— graph.py	# Lop Graph, 3 cach bieu dien, doc file
— traversal.py	# BFS, DFS
— bipartite.py	# Kiem tra do thi hai phia
— shortest_path.py	# Dijkstra, Bellman-Ford
— euler.py	# Kiem tra dieu kien Euler, Fleury, Hierholzer
— mst.py	# Prim, Kruskal, Union-Find
— route_orchestrator.py	# Ham dieu phoi (Phan 8)
— visualize.py	# Truc quan hoa (matplotlib/networkx)
— real_world.py	# Vi du ung dung thuc te (Phan 8)
— main.py	# Chuong trinh demo tong hop
— samples/	# Cac file do thi mau (8-10 dinh) cho tung thuat toan
— outputs/	# Hinh anh + log ket qua chay demo

Định dạng file input đồ thị (text, tự định nghĩa):

```
n_dinh
directed|undirected
weighted|unweighted
labels: A B C D ...
u1 v1 [w1]
u2 v2 [w2]
...
```

Cách chạy chương trình demo tổng hợp:

```
012012204402_Group_12/src
python3 main.py
```

Toàn bộ log kết quả chạy được lưu tại outputs/run_log.txt, hình ảnh (.png) lưu trong outputs/.

3. Nhập đồ thị, vẽ và lưu hình

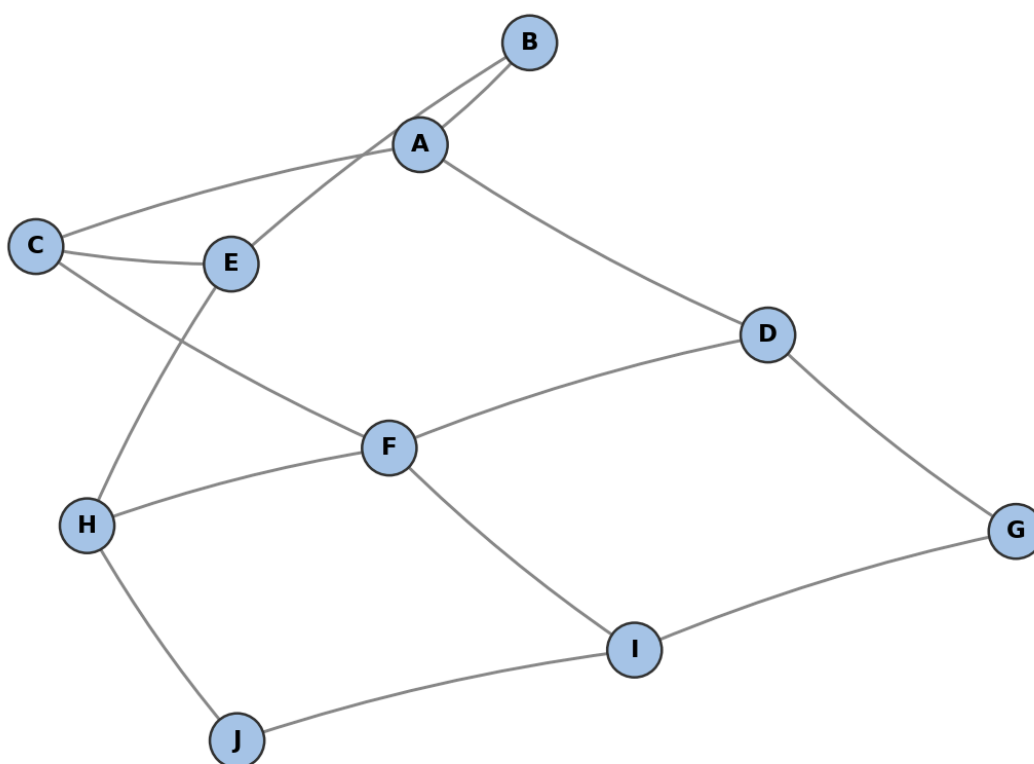
Đồ thị được nhập từ file text (/Samples). Sau khi nhập, chương trình gọi module visualize.py để vẽ đồ thị và lưu thành file ảnh .png.

Đồ thị mẫu (samples/basic_undirected.txt) — 10 đỉnh, 14 cạnh

cấu trúc có nhiều nhánh rẽ và 2 chu trình lồng nhau:

Canh: A-B, A-C, A-D, B-E, C-E, C-F, D-F, D-G, E-H, F-H, F-I, G-I, H-J, I-J

Đồ thị đầu vào (vô hướng)



Hình 1 – Đồ thị đầu vào 10 đỉnh được vẽ và lưu tự động

4. Các phương pháp biểu diễn đồ thị

Từ cùng một đối tượng Graph, chương trình chuyển đổi qua lại giữa 3 cách biểu diễn: Ma trận kề, Danh sách kề, Danh sách cạnh. Áp dụng trên đồ thị 10 đỉnh ở Phần 1:

Ma trận kề (Adjacency Matrix)

	A	B	C	D	E	F	G	H	I	J
A	0	1	1	1	0	0	0	0	0	0
B	1	0	0	0	1	0	0	0	0	0
C	1	0	0	0	1	1	0	0	0	0
D	1	0	0	0	0	1	1	0	0	0
E	0	1	1	0	0	0	0	1	0	0
F	0	0	1	1	0	0	0	1	1	0
G	0	0	0	1	0	0	0	0	1	0
H	0	0	0	0	1	1	0	0	0	1
I	0	0	0	0	0	1	1	0	0	1
J	0	0	0	0	0	0	0	1	1	0

Danh sách kề (Adjacency List)

A: B, C, D
B: A, E
C: A, E, F
D: A, F, G
E: B, C, H
F: C, D, H, I
G: D, I
H: E, F, J
I: F, G, J
J: H, I

Danh sách cạnh (Edge List)

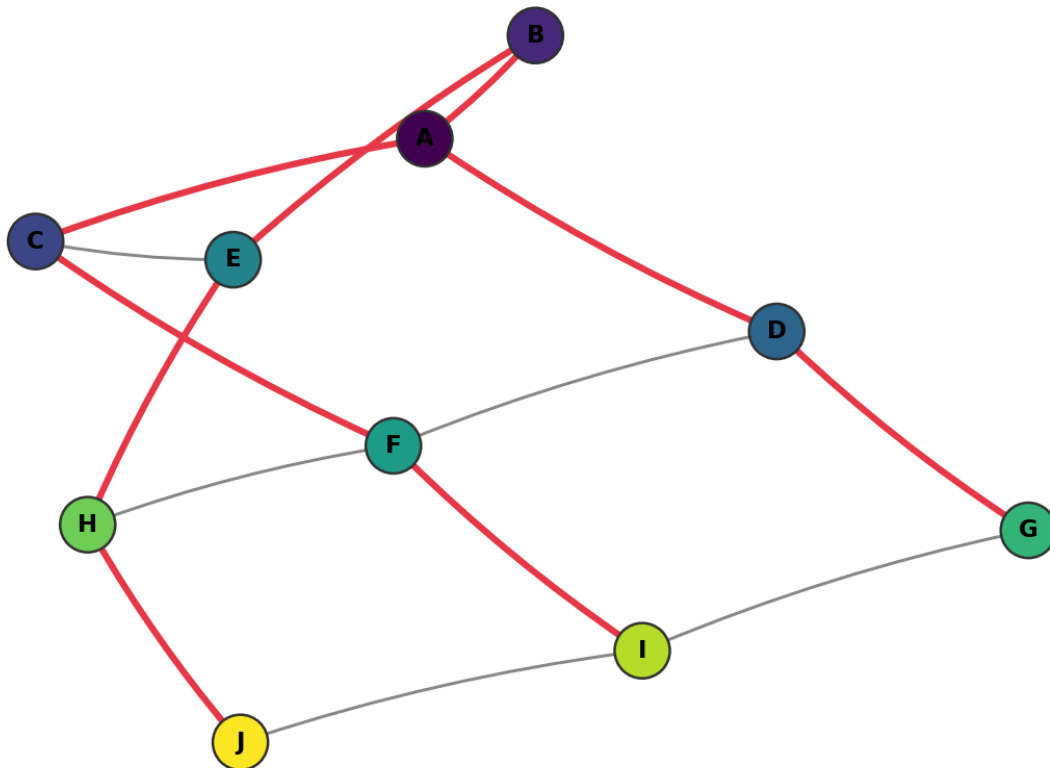
(A-B),(A-C),(A-D),(B-E),(C-E),(C-F),(D-F),(D-G),(E-H),(F-H),(F-I),(G-I),(H-J),(I-J)

Chương trình đã kiểm chứng chiều chuyển đổi ngược: từ Ma trận kề dựng lại Graph (Graph.from_adjacency_matrix) và từ Danh sách cạnh dựng lại Graph (Graph.from_edge_list) – danh sách cạnh sau tái tạo trùng khớp hoàn toàn với đồ thị gốc, xác nhận phép chuyển đổi 2 chiều giữa 3 cách biểu diễn là chính xác.

5. Duyệt đồ thị BFS & DFS

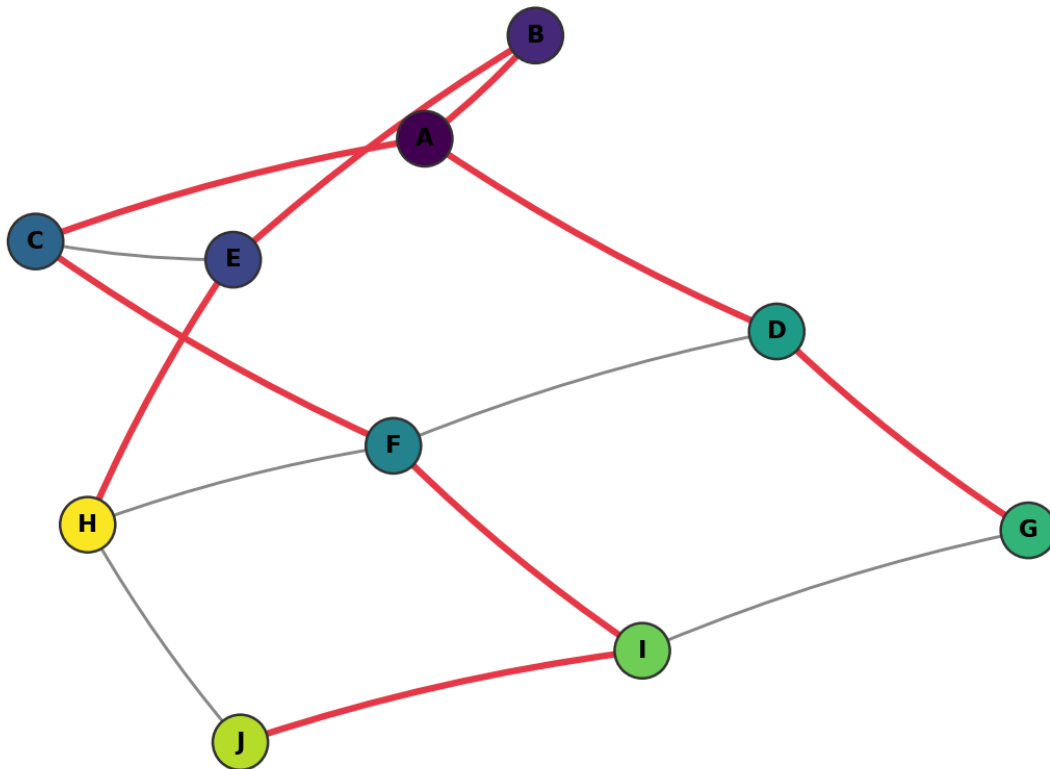
BFS dùng hàng đợi tự cài đặt (list + con trỏ head). DFS cài đặt cả 2 phiên bản: đệ quy và lặp dùng ngăn xếp tự cài đặt. Tại mỗi đỉnh, các đỉnh kề được duyệt theo thứ tự chỉ số “tăng dần” để kết quả tất định, dễ đối chiếu tay.

Do thi duyệt BFS từ A



Hình 2 – Cây duyệt BFS từ A

Do thi duyệt DFS từ A



Hình 3 – Cây duyệt DFS từ A

Đối chiếu với kết quả chạy tay

Danh sách kề: A:[B,C,D] B:[A,E] C:[A,E,F] D:[A,F,G] E:[B,C,H] F:[C,D,H,I] G:[D,I] H:[E,F,J] I:[F,G,J] J:[H,I]

Thuật toán	Kết quả chạy tay	Kết quả chương trình	Khớp?
BFS(A)	A,B,C,D,E,F,G,H,I,J	A→B→C→D→E→F→G→H→I→J	Đúng
DFS(A)	A,B,E,C,F,D,G,I,J,H	A→B→E→C→F→D→G→I→J→H	Đúng

Diễn giải chạy tay BFS(A)

Mức 0: A. Mức 1 (láng giềng của A theo thứ tự tăng dần): B, C, D. Mức 2: xét lần lượt $B \rightarrow$ thêm E; $C \rightarrow$ E đã có, thêm F; $D \rightarrow$ F đã có, thêm G. Mức 3: $E \rightarrow$ thêm H; $F \rightarrow$ H đã có (từ E), thêm I; $G \rightarrow$ D, I đã có. Mức 4: $H \rightarrow$ thêm J; $I \rightarrow$ F, G, J đã có. Kết quả: A, B, C, D, E, F, G, H, I, J.

Diễn giải chạy tay DFS(A)

Từ A đi sâu theo đỉnh nhỏ nhất $B \rightarrow$ từ B chỉ còn E mới $\rightarrow$ từ E, đỉnh nhỏ nhất chưa thăm là C $\rightarrow$ từ C, còn F mới $\rightarrow$ từ F, đỉnh nhỏ nhất chưa thăm là D (ưu tiên trước H, I) $\rightarrow$ từ D, còn G mới $\rightarrow$ từ G, còn I mới $\rightarrow$ từ I, còn J mới $\rightarrow$ từ J, còn H mới $\rightarrow$ từ H không còn gì mới. Kết quả: A, B, E, C, F, D, G, I, J, H — trùng khớp hoàn toàn với chương trình.

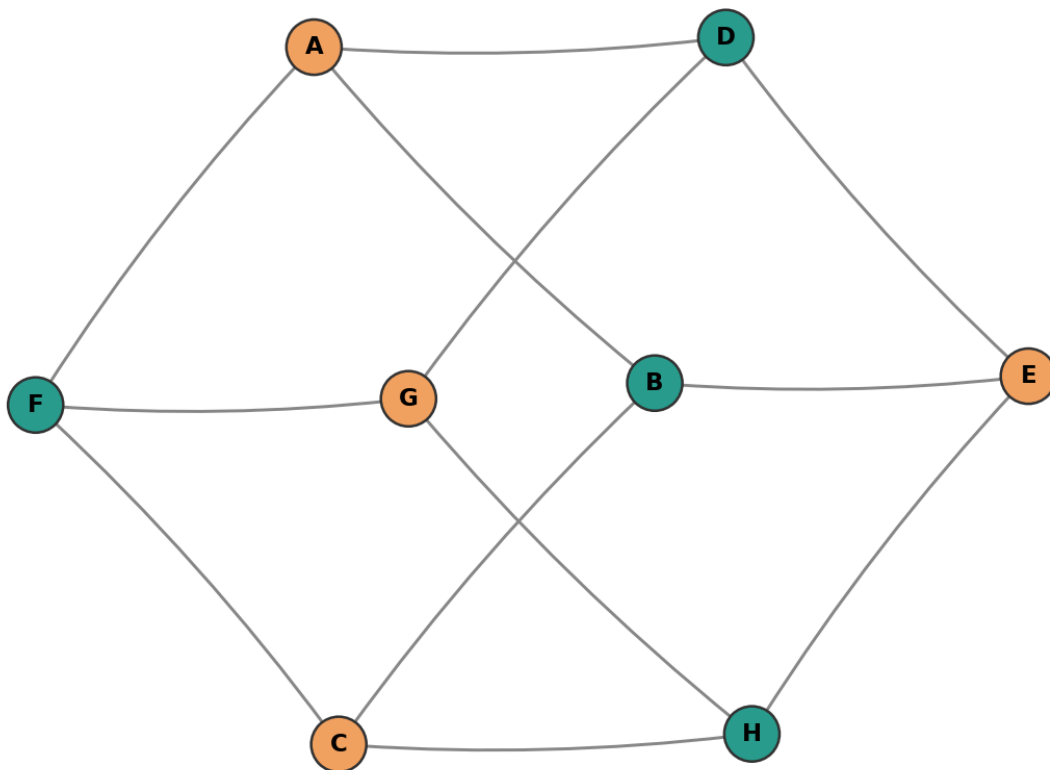
6. Kiểm tra đồ thị hai phía (Bipartite)

Thuật toán: tô màu 2 màu bằng BFS trên toàn bộ các thành phần liên thông. Nếu tồn tại cạnh nối 2 đỉnh cùng màu $\Rightarrow$ đồ thị KHÔNG hai phía.

Trường hợp 1: Đồ thị hai phía (8 đỉnh, 12 cạnh)

Cạnh: A-B, A-D, A-F, C-B, C-F, C-H, E-D, E-H, E-B, G-F, G-H, G-D

Đồ thị hai phía (2 màu)



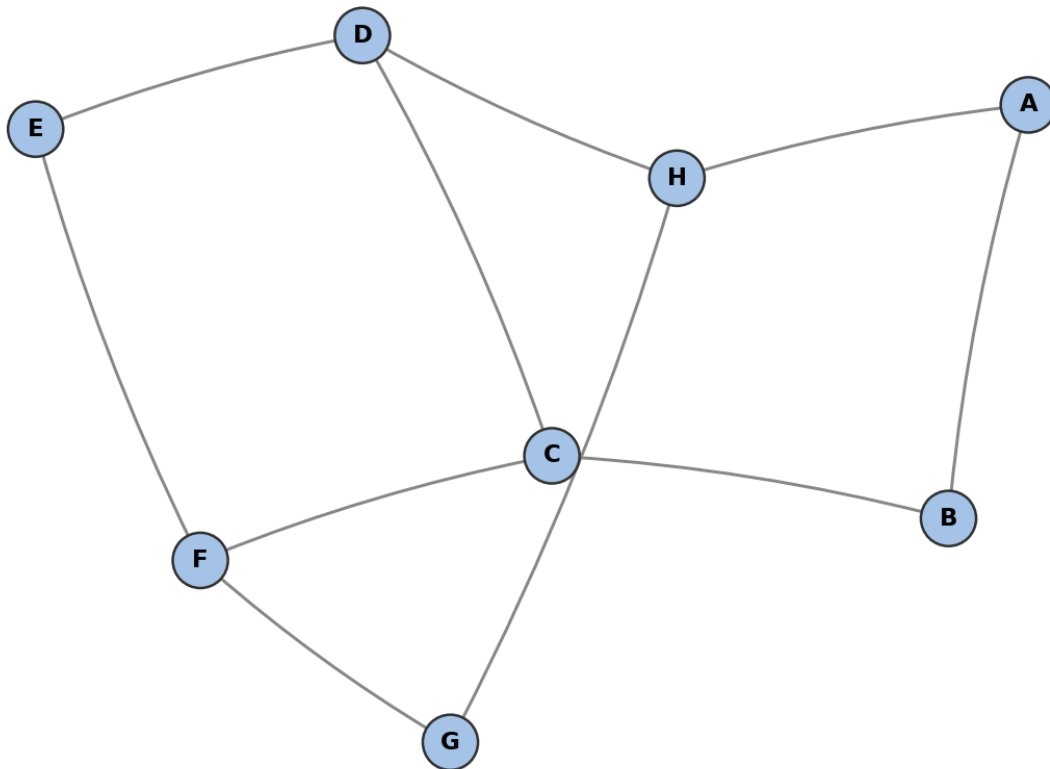
Hình 4 – Tập $\{A,C,E,G\}$ (cam) và $\{B,D,F,H\}$ (xanh)

Chạy tay: tô $A=0 \rightarrow B,D,F=1$ (kề A). Từ B: $C=0, E=0$ (kề B). Từ D: $E=0$ (trùng), $G=0$. Từ F: $C=0$ (trùng), $G=0$ (trùng). Kiểm tra mọi cạnh còn lại (C-H, E-H, G-H): H phải màu 1 (kề $C=0, E=0, G=0$) – nhất quán ở cả 3 cạnh. Kết quả: $\{A,C,E,G\}$ và $\{B,D,F,H\}$, không có cạnh nào nối 2 đỉnh cùng màu $\Rightarrow$ ĐÚNG là đồ thị hai phía — trùng khớp với chương trình.

Trường hợp 2: Đồ thị không hai phía (8 đỉnh, 10 cạnh)

Cạnh: A-B, B-C, C-D, D-E, E-F, F-C, F-G, G-H, H-A, D-H

Đồ thị không hai phía (có chu trình lẻ)



Hình 5 – Đồ thị chứa chu trình lẻ ẩn bên trong

Chạy tay: tô $A=0 \rightarrow B=1, H=1$. Từ B: $C=0$. Từ C: $D=1, F=1$ (kề C). Từ D: $E=0$ (kề D), nhưng D cũng kề H (đã tô 1) $\Rightarrow D=1$ mâu thuẫn với D vừa tô... thực tế thuật toán phát hiện mâu thuẫn ngay khi xét cạnh D-H: D được tô màu 1 (qua C), H đã tô màu 1 (qua A) $\Rightarrow$ 2 đỉnh cùng màu có cạnh nối $\Rightarrow$ Mâu thuẫn. Nguyên nhân sâu xa: chu trình C-D-H-A-B-C có độ dài 5 (lẻ) ẩn trong đồ thị. Kết quả: không hai phía — trùng khớp với chương trình.

7. Đường đi ngắn nhất: Dijkstra & Bellman–Ford

Cả 2 thuật toán tự cài đặt, không dùng heapq. Dijkstra chọn đỉnh có dist nhỏ nhất trong các đỉnh chưa thăm bằng duyệt tuyến tính $O(V)$ mỗi bước – chỉ đúng khi trọng số không âm. Bellman–Ford lặp thư giãn tất cả cạnh tối đa $V-1$ lần, cho phép trọng số âm và phát hiện chu trình âm.

7.1 Dijkstra (9 đỉnh, 16 cạnh, trọng số không âm)

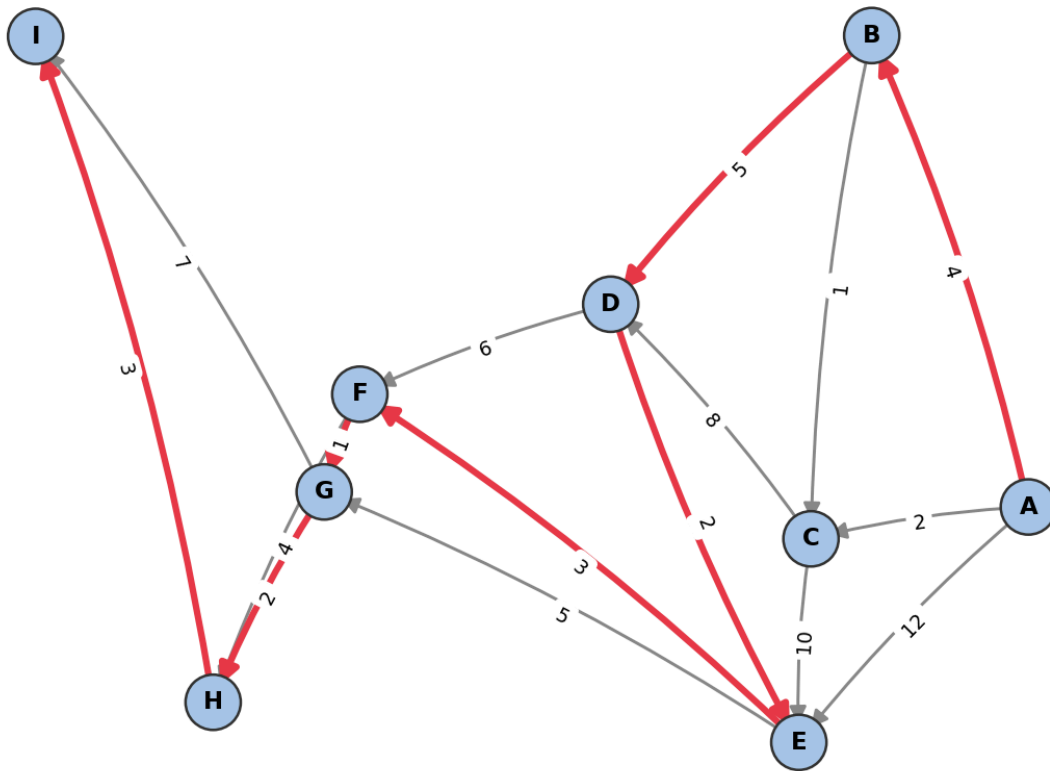
A->B(4) A->C(2) A->E(12) B->C(1) B->D(5) C->D(8) C->E(10)
D->E(2) D->F(6) E->F(3) E->G(5) F->G(1) F->H(2) F->I(7) G->H(3) G->I(4) H->I(3)

Diễn biến (chọn đỉnh dist nhỏ nhất, thư giãn các cạnh đi ra):

Bước	Đỉnh chọn	dist	Cập nhật đỉnh kề
1	A	0	B=4, C=2, E=12
2	C	2	D=10 (qua C)
3	B	4	D=9 (qua B, tốt hơn 10)
4	D	9	E=11 (qua D, tốt hơn 12), F=15
5	E	11	F=14 (tốt hơn 15), G=16
6	F	14	G=15 (tốt hơn 16), H=18
7	G	15	H=17 (tốt hơn 18), I=22
8	H	17	I=20 (tốt hơn 22)
9	I	20	—

Kết quả: $\text{dist}(I) = 20$, đường đi $A \rightarrow B \rightarrow D \rightarrow E \rightarrow F \rightarrow G \rightarrow H \rightarrow I$.

Dijkstra: đường ngắn nhất A→I (do dài=20)



Hình 6 – Đường đi ngắn nhất Dijkstra $A \rightarrow I$

Đối chiếu chạy tay: thực hiện đúng trình tự thư giãn cạnh như bảng trên cho ra CÙNG kết quả $\text{dist}=20$ và cùng đường đi $\Rightarrow$ TRÙNG KHỚP với chương trình (xem log outputs/run_log.txt).

7.2 Bellman–Ford (8 đỉnh, 14 cạnh, có cạnh âm, không có chu trình âm)

A→B(6) A→C(4) B→D(5) B→C(-2) C→D(3) C→E(6)
D→F(4) D→E(-3) E→F(2) E→G(5) F→H(-1) G→F(2) E→H(9)

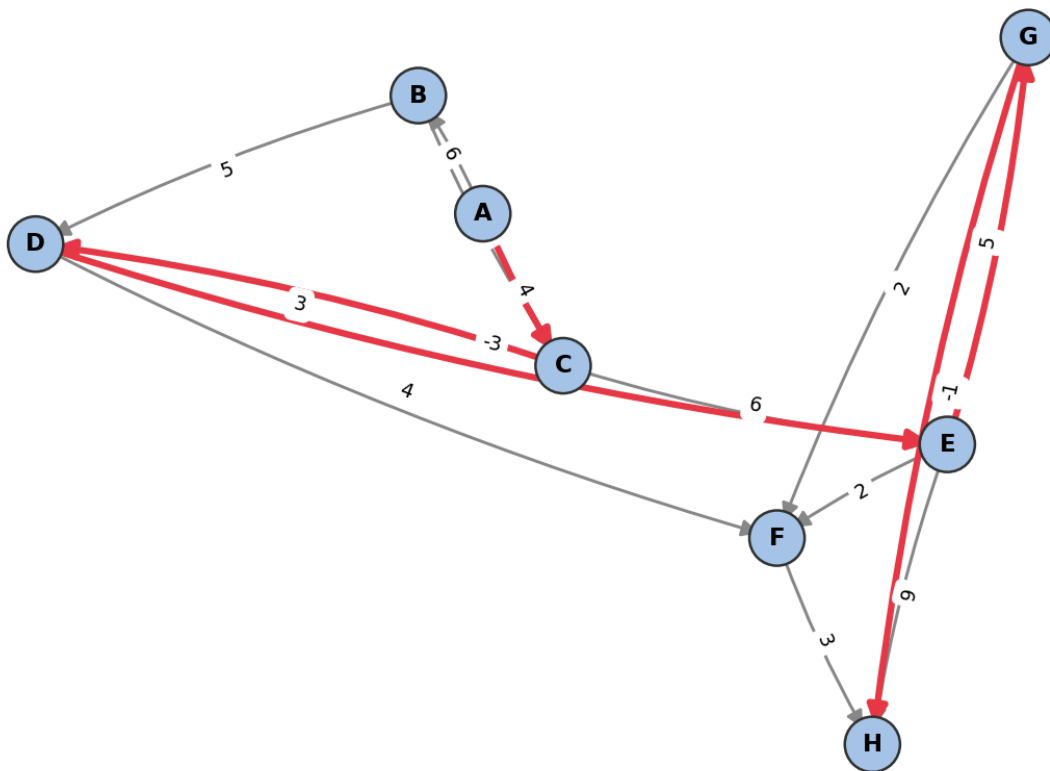
Chạy tay – lặp thư giãn toàn bộ cạnh theo thứ tự liệt kê ở trên:

Vòng lặp	dist[A,B,C,D,E,F,G,H]
Khởi tạo	0, ∞, ∞, ∞, ∞, ∞, ∞, ∞
Vòng 1	0, 6, 4, 7, 4, 6, 9, 8
Vòng 2 (không đổi)	0, 6, 4, 7, 4, 6, 9, 8 → dừng sớm

Diễn giải: $\text{dist}(C) = \min(4, \text{dist}(A) + 4 = 4) \rightarrow 4$; $\text{dist}(D) = \text{dist}(C) + 3 = 7$; $\text{dist}(E) = \min(\text{dist}(C) + 6 = 10, \text{dist}(D) + (-3) = 4) = 4$; $\text{dist}(F) = \min(\text{dist}(D) + 4 = 11, \text{dist}(E) + 2 = 6) = 6$; $\text{dist}(G) = \text{dist}(E) + 5 = 9$; $\text{dist}(H) = \min(\text{dist}(F) + 3 = 9, \text{dist}(G) + (-1) = 8, \text{dist}(E) + 9 = 13) = 8$. Không có chu trình âm (hội tụ ngay từ vòng lặp 2, không cần đủ $V-1=7$ vòng).

Kết quả: $\text{dist}(H) = 8$, đường đi $A \rightarrow C \rightarrow D \rightarrow E \rightarrow G \rightarrow H$.

Bellman-Ford: đường ngắn nhất A→H (độ dài=8)



Hình 7 – Đường đi ngắn nhất Bellman-Ford $A \rightarrow H$

Đối chiếu chạy tay: kết quả tính tay $\text{dist}=8$, đường đi A-C-D-E-G-H TRÙNG KHỚP hoàn toàn với chương trình.

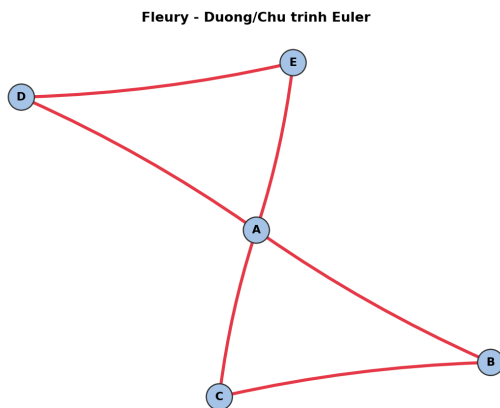
8. Thuật toán Fleury và Hierholzer

Đường đi Euler đi qua tất cả các cạnh đúng 1 lần. Chu trình Euler là đường đi Euler bắt đầu/kết thúc cùng 1 đỉnh. Điều kiện tồn tại (đồ thị vô hướng, liên thông trên các đỉnh bậc > 0): Chu trình Euler khi mọi đỉnh bậc chẵn; Đường đi Euler khi có đúng 2 đỉnh bậc lẻ.

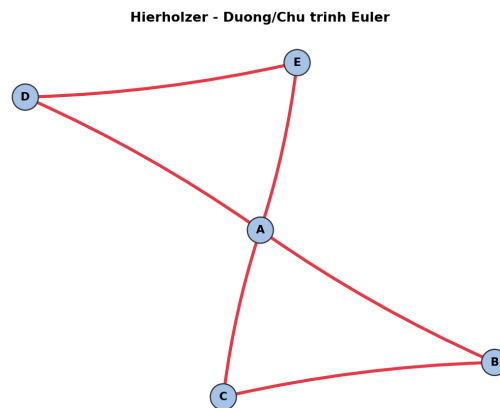
- Fleury: ưu tiên đi qua cạnh không phải cầu (bridge); chỉ đi qua cầu khi bắt buộc (hàm `_is_bridge()` so sánh số đỉnh liên thông trước/sau khi tạm xoá cạnh).
- Hierholzer: hiệu quả hơn ($O(E)$), dùng ngăn xếp, khi kết thì ghi nhận đỉnh vào chu trình kết quả rồi quay lui.

Trường hợp có chu trình Euler (2 tam giác chung đỉnh A)

Cạnh: A-B, B-C, C-A, A-D, D-E, E-A. Bậc: A=4, B=C=D=E=2 (đều chẵn) $\Rightarrow$ tồn tại chu trình Euler.



Fleury: $A \rightarrow B \rightarrow C \rightarrow A \rightarrow D \rightarrow E \rightarrow A$



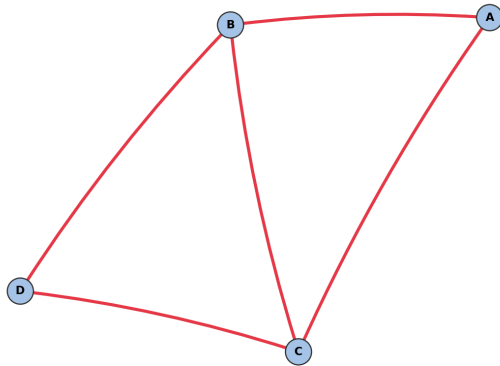
Hierholzer: $A \rightarrow E \rightarrow D \rightarrow A \rightarrow C \rightarrow B \rightarrow A$

Cả 2 thuật toán đều tìm được chu trình đi qua đủ 6 cạnh và quay lại đỉnh xuất phát — khớp lý thuyết.

Trường hợp có đường đi Euler (không phải chu trình)

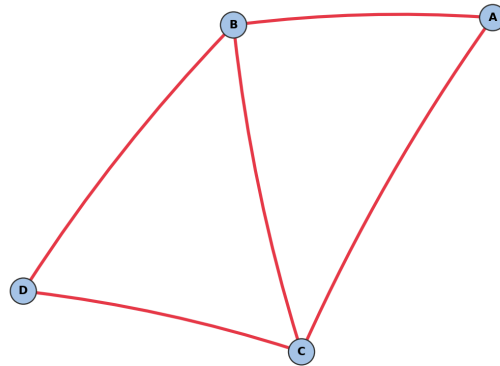
Cạnh: A-B, B-C, C-A, C-D, D-B. Bậc: A=2, B=3, C=3, D=2 $\rightarrow$ đúng 2 đỉnh bậc lẻ (B, C) $\Rightarrow$ tồn tại đường đi Euler bắt đầu/kết thúc tại B hoặc C.

Fleury - Duong/Chu trình Euler



Fleury: $B \rightarrow A \rightarrow C \rightarrow B \rightarrow D \rightarrow C$

Hierholzer - Duong/Chu trình Euler



Hierholzer: $B \rightarrow D \rightarrow C \rightarrow A \rightarrow B \rightarrow C$

Cả 2 kết quả đều bắt đầu tại B, kết thúc tại C (2 đỉnh bậc lẻ), đi qua đủ 5 cạnh — đúng lý thuyết dự đoán. Lưu ý: 2 ví dụ này được giữ ở quy mô nhỏ, gọn vì mục đích minh họa trực tiếp điều kiện lý thuyết; trường hợp đồ thị thực tế lớn hơn, không thỏa điều kiện Euler, được xử lý riêng ở Phần 8 bằng bài toán Người phát thư Trung Quốc.

9. Thuật toán Prim và Kruskal

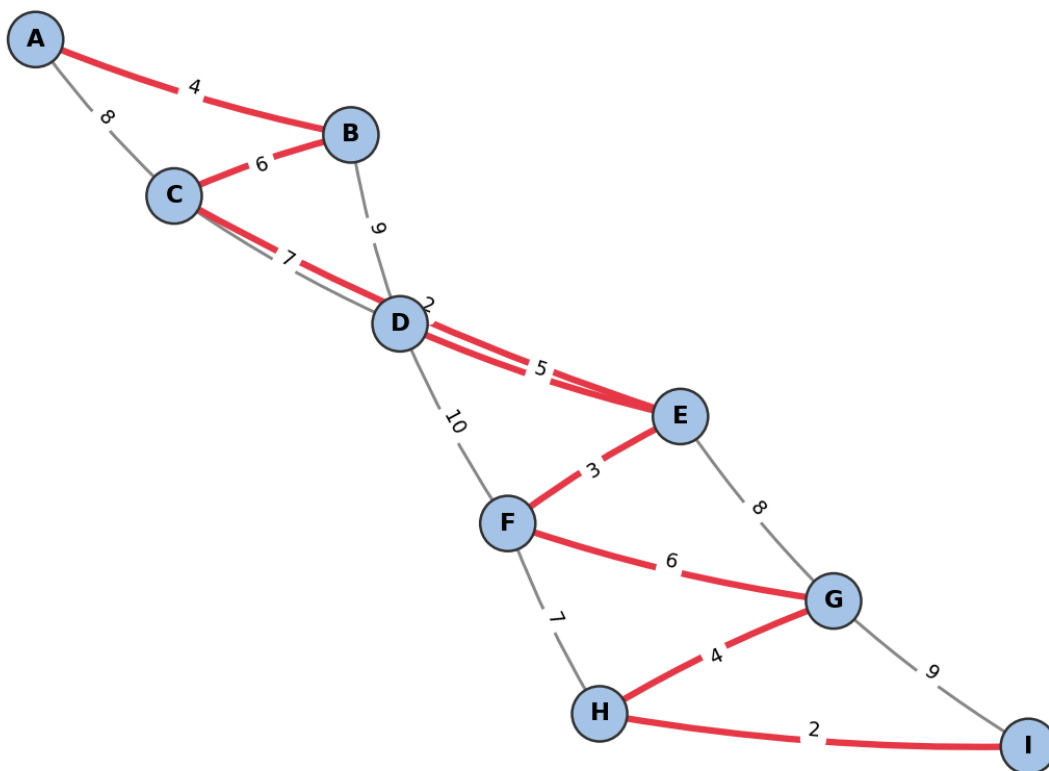
Cây khung nhỏ nhất (MST) là cây con của đồ thị vô hướng có trọng số, đi qua tất cả đỉnh với tổng trọng số cạnh nhỏ nhất.

- Prim: từ 1 đỉnh, mỗi bước thêm cạnh nhỏ nhất nối 1 đỉnh trong cây với 1 đỉnh ngoài cây (mảng `key[]`, chọn tuyến tính $O(V)$).
- Kruskal: sắp xếp cạnh tăng dần, thêm cạnh nếu không tạo chu trình (Union-Find tự cài, có path compression + union-by-rank).

Đồ thị mẫu (`mst_graph.txt`) — 9 đỉnh, 15 cạnh có trọng số:

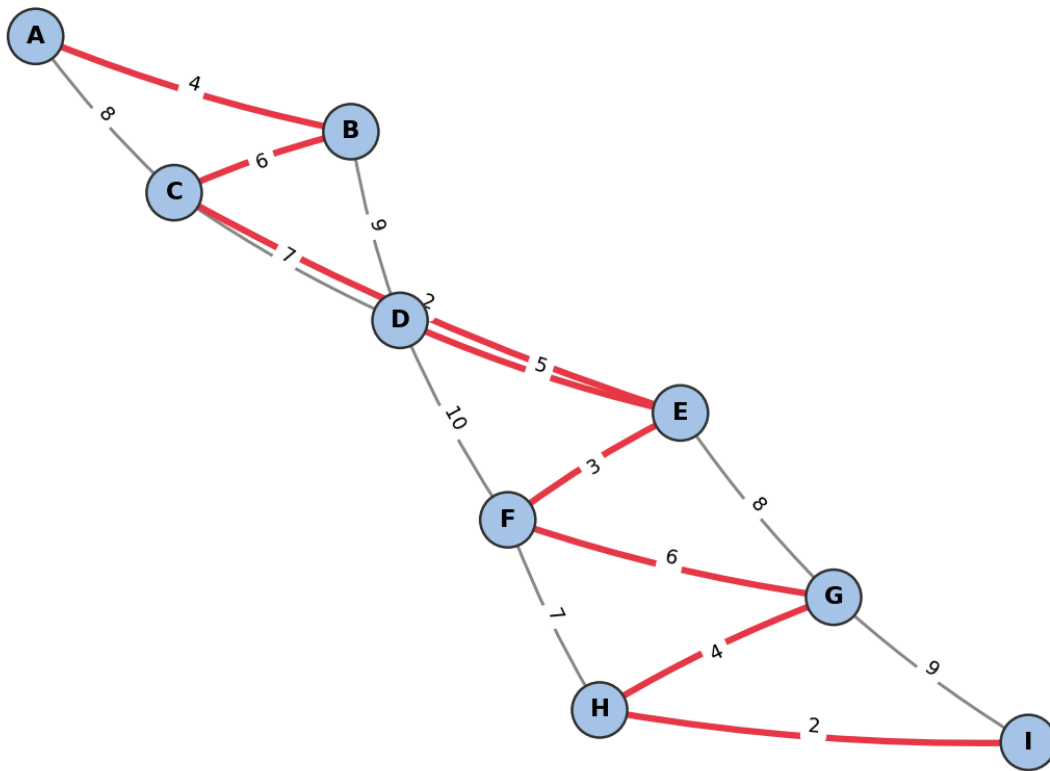
```
A-B(4) A-C(8) B-C(6) B-D(9) C-E(2) C-D(7) D-E(5) D-F(10)
E-F(3) E-G(8) F-G(6) F-H(7) G-H(4) G-I(9) H-I(2)
```

Prim - Cây khung nhỏ nhất (tổng = 32)



Hình 9.1 – MST theo Prim (tổng = 32)

Kruskal - Cây khung nhỏ nhất (tổng = 32)



Hình 9.2 – MST theo Kruskal (tổng = 32)

Diễn biến Kruskal (chạy tay, sắp xếp cạnh tăng dần)

Danh sách cạnh sắp xếp tăng dần: C-E(2), H-I(2), E-F(3), A-B(4), G-H(4), D-E(5), B-C(6), F-G(6), C-D(7), F-H(7), A-C(8), E-G(8), B-D(9), G-I(9), D-F(10)

Cạnh (w)	Quyết định	Lý do
C-E (2)	Chọn	Nối 2 thành phần riêng biệt {C}, {E}
H-I (2)	Chọn	Nối 2 thành phần riêng biệt {H}, {I}
E-F (3)	Chọn	Nối {C,E} với {F}
A-B (4)	Chọn	Nối {A} với {B}
G-H (4)	Chọn	Nối {G} với {H,I}
D-E (5)	Chọn	Nối {D} với {C,E,F}
B-C (6)	Chọn	Nối {A,B} với {C,D,E,F}
F-G (6)	Chọn	Nối {A,B,C,D,E,F} với {G,H,I} → đủ 8 cạnh ($n-1=8$), DỪNG

Tổng trọng số MST (chạy tay) = $2+2+3+4+4+5+6+6 = 32$, trùng khớp với cả Prim và Kruskal của chương trình — đúng lý thuyết (Prim và Kruskal trên cùng 1 đồ thị liên thông luôn cho cùng tổng trọng số MST).

10. Ứng dụng thực tế: Tối ưu lộ trình xe quét rác

10.1 Mô tả bài toán

Một công ty vệ sinh đô thị có 1 xe quét rác cần đi qua tất cả các con đường trong 1 khu phố để quét dọn/thu gom rác 2 bên đường. Đây là điểm khác biệt cốt lõi so với bài toán giao hàng (chỉ cần đến từng điểm) — bài toán quét rác cần đi qua hết mọi đoạn đường, tức là một bài toán trên cạnh(edge) của đồ thị chứ không phải trên đỉnh (vertex). Xe xuất phát từ bãi đỗ (garage) và phải quay về đúng garage sau khi hoàn thành, sao cho tốn ít quãng đường di chuyển thừa nhất có thể.

10.2 Mô hình hoá thành bài toán đồ thị

Thành phần bài toán thực tế	Tương ứng trong mô hình đồ thị
Mỗi giao lộ (ngã ba/ngã tư) trong khu phố	Một đỉnh (node)
Mỗi đoạn đường cần quét	Một cạnh (edge) vô hướng
Độ dài đoạn đường (m)	Trọng số của cạnh
Bãi đỗ xe (garage)	Đỉnh xuất phát = đỉnh kết thúc
Cần tìm chu trình đi qua mọi đoạn đường đúng 1 lần, tốn ít quãng đường thừa nhất	Bài toán chu trình Euler mở rộng.

10.3 Vấn đề mấu chốt: mạng lưới thực tế không thỏa điều kiện Euler

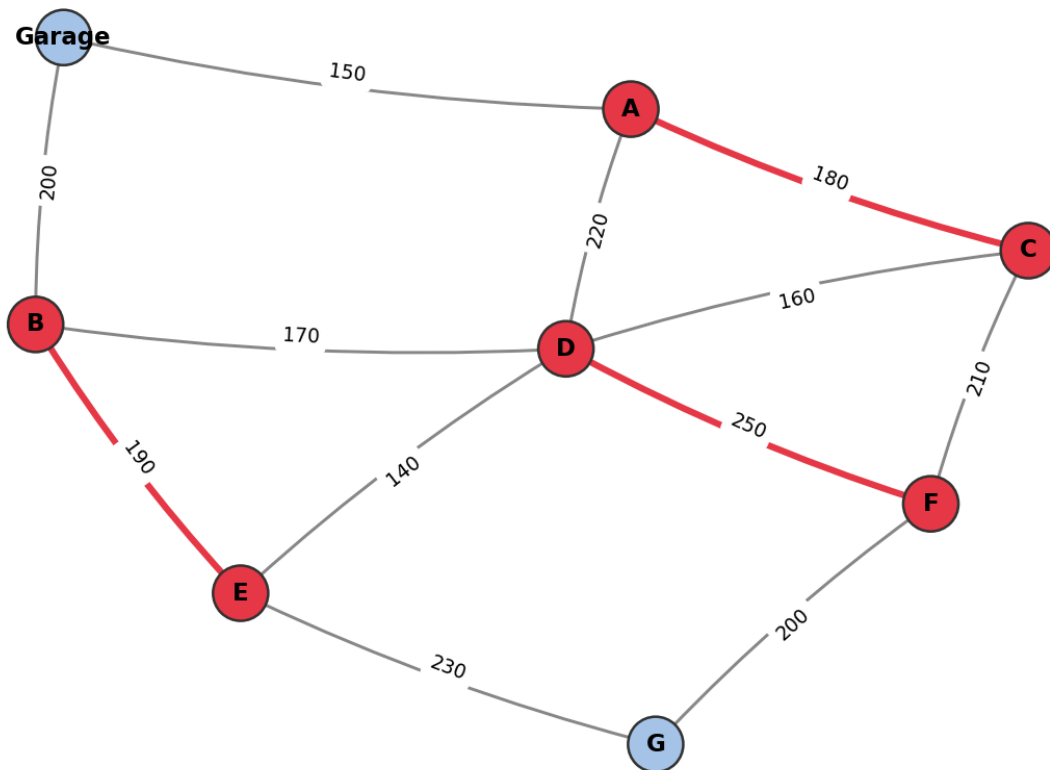
Đây chính là điểm khác biệt quan trọng nhất so với các ví dụ ở Phần 7.1–7.2 (đồ thị được thiết kế sẵn để có đúng 0 hoặc 2 đỉnh bậc lẻ). Với một khu phố “thật”, hầu như không thể đảm bảo mọi giao lộ đều có bậc chẵn. Mạng lưới mô phỏng dưới đây gồm 8 giao lộ, 12 đoạn đường:

Garage-A(150) Garage-B(200) A-C(180) A-D(220) B-D(170) B-E(190)
C-D(160) C-F(210) D-E(140) D-F(250) E-G(230) F-G(200)

Giao lộ	Bậc	Chẵn/Lẻ
Garage	2	Chẵn
A	3	Lẻ
B	3	Lẻ
C	3	Lẻ
D	5	Lẻ
E	3	Lẻ
F	3	Lẻ
G	2	Chẵn

Kết quả: 6/8 giao lộ có bậc lẻ (A, B, C, D, E, F) — vi phạm hoàn toàn điều kiện tồn tại chu trình Euler (chỉ chấp nhận 0 đỉnh bậc lẻ). Fleury và Hierholzer không thể áp dụng trực tiếp. Đây chính là phần "nâng cao thật sự" mà đề bài yêu cầu: xử lý khi đồ thị không thỏa điều kiện Euler.

Ghép cặp đỉnh bậc lẻ (do) và cạnh nhân đôi do ghép cặp (do dam)



Hình 10.1 – Mạng lưới đường phố mô phỏng (Garage = bãi đỗ xe)

10.4 Thuật toán xử lý: Bài toán Dijkstra, Ghép cặp trọng số nhỏ nhất và Hierholzer

Ý tưởng : module `route_orchestrator.py` — đóng vai trò là hàm điều phối, gọi lại 2 thuật toán Dijkstra đã cài ở Phần 5 và Hierholzer ở Phần 8:

- Bước 1: Tìm tất cả đỉnh bậc lẻ (số lượng luôn là số chẵn, theo bổ đề bắt tay).
- Bước 2: Tính khoảng cách đường đi ngắn nhất (dùng Dijkstra) giữa mọi cặp đỉnh bậc lẻ.
- Bước 3: Tìm phép ghép cặp hoàn hảo trọng số nhỏ nhất (Minimum Weight Perfect Matching) trên tập đỉnh bậc lẻ, dùng khoảng cách ngắn nhất vừa tính làm trọng số — cài đặt bằng quy hoạch đệ quy (backtracking có cắt nhánh), khả thi vì số đỉnh bậc lẻ trong thực tế thường nhỏ.
- Bước 4: Với mỗi cặp đã ghép, nhân đôi (thêm cạnh ảo) các cạnh nằm trên đường đi ngắn nhất giữa 2 đỉnh đó. Sau bước này, mọi đỉnh của đồ thị đã tăng cường đều có bậc chẵn.

- Bước 5: Chạy Hierholzer trên đồ thị đã tăng cường để tìm chu trình Euler — chính là lộ trình tối ưu cho xe quét rác.

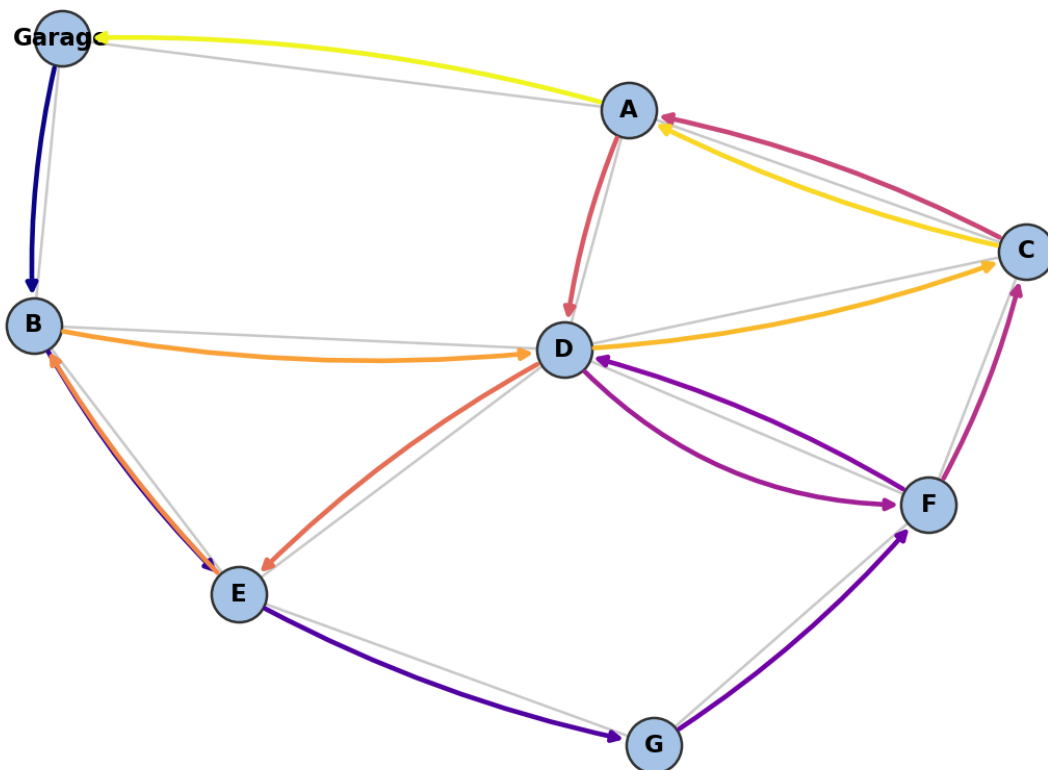
10.5 Kết quả chạy chương trình

Bước 2 và 3 — khoảng cách ngắn nhất giữa các cặp đỉnh bậc lẻ và phương án ghép cặp tối ưu tìm được:

Cặp ghép	Khoảng cách ngắn nhất (m)
A ↔ C	180
B ↔ E	190
D ↔ F	250

Tổng quãng đường đi lại phát sinh = $180 + 190 + 250 = 620\text{m}$. Đây là phương án ghép cặp có tổng chi phí nhỏ nhất trong toàn bộ 15 cách ghép cặp khả dĩ của 6 đỉnh bậc lẻ (chương trình duyệt toàn bộ bằng backtracking).

Lộ trình tối ưu (Dijkstra + Ghep cap + Hierholzer), tổng = 2920 m

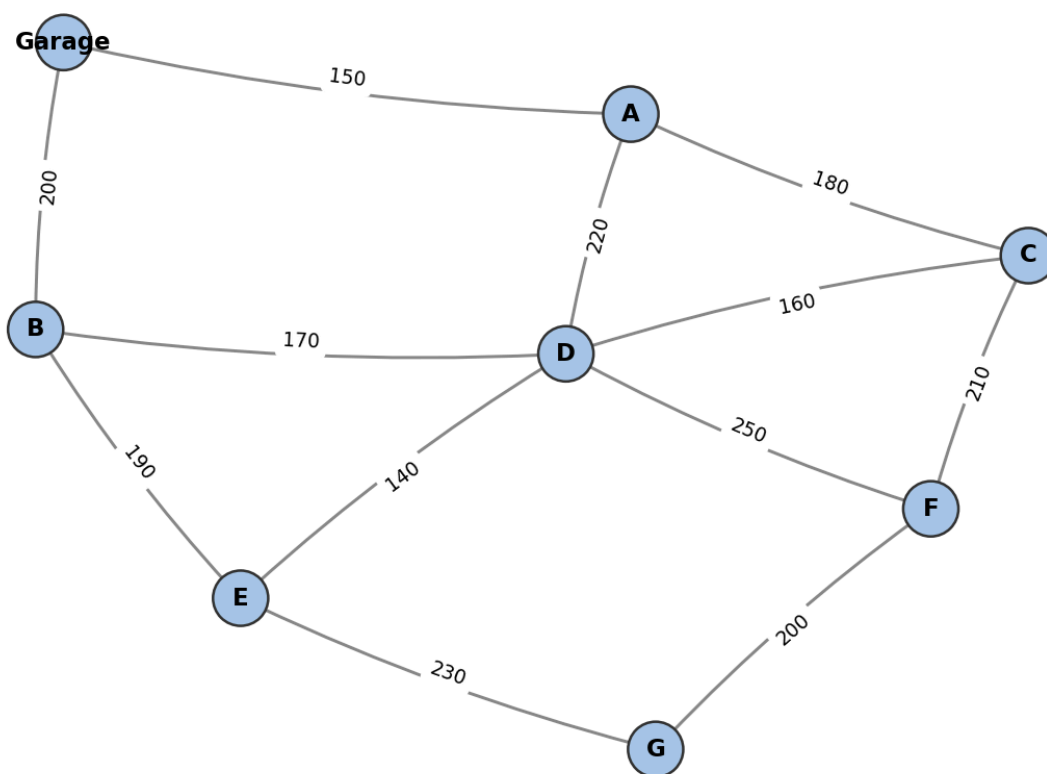


Hình 10.2 – Đỉnh bậc lẻ (đỏ) và các cạnh bị nhân đôi do ghép cặp (đỏ đậm)

Sau khi nhân đôi 3 cặp cạnh trên, chạy Hierholzer trên đồ thị đã tăng cường (15 cạnh, mọi đỉnh đều bậc chẵn) cho ra lộ trình Euler hoàn chỉnh:

Garage -> B -> E -> G -> F -> D -> F -> C -> A -> D -> E -> B -> D -> C -> A -> Garage

Mạng lưới đường phố khu vực (Garage = điểm xuất phát)



Hình 10.3 – Lộ trình tối ưu cho xe quét rác (màu chuyển dần theo thứ tự đi qua)

Dại lượng	Giá trị
Tổng độ dài đường cần quét (mỗi đoạn tính 1 lần)	2.300 m
Quãng đường phải đi lại thêm (deadhead, do ghép cặp)	620 m
TỔNG QUÃNG ĐƯỜNG XE PHẢI DI CHUYỂN	2.920 m

Ý nghĩa thực tiễn: nếu công ty chỉ đơn giản cho xe đi ngẫu nhiên qua từng đoạn đường mà không tính toán, quãng đường đi lại dư thừa có thể lớn hơn nhiều so với 620m tối ưu này. Với quy mô một khu phố lớn có hàng chục, hàng trăm giao lộ, việc tự tính tay phương án ghép cặp tối ưu là bất khả thi — đây chính là giá trị thực sự của việc áp dụng thuật toán (kết hợp Dijkstra + ghép cặp trọng số nhỏ nhất + Hierholzer) vào bài toán vận hành đội xe vệ sinh đô thị trong thực tế.

11. Kết luận

Nhóm đã hoàn thành đầy đủ yêu cầu đề bài: xây dựng cấu trúc Graph tổng quát; cài đặt và kiểm chứng bằng tay 3 phương pháp biểu diễn; BFS, DFS, kiểm tra hai phía, Dijkstra, Bellman-Ford (Phần cơ bản); Fleury, Hierholzer, Prim, Kruskal kèm trực quan hoá (Phần nâng cao); và áp dụng Euler mở rộng (kết hợp Dijkstra, ghép cặp trọng số nhỏ nhất, Hierholzer) vào bài toán thực tế xe quét rác đô thị, xử lý đúng trường hợp "khó" mà giảng viên yêu cầu — đồ thị không thỏa điều kiện Euler ngay từ đầu.

Theo góp ý của giảng viên, toàn bộ đồ thị mẫu đã được nhóm tự thiết kế lại với quy mô 8-10 đỉnh, không thể suy ra đáp án bằng trực giác, đòi hỏi chạy đúng thuật toán (hoặc tính tay đầy đủ từng bước) mới ra kết quả. Toàn bộ thuật toán được tự cài đặt bằng cấu trúc dữ liệu Python thuần, chỉ dùng matplotlib/networkx cho mục đích trực quan hoá như yêu cầu đề bài.

Hướng phát triển thêm: cài đặt heap nhị phân tự viết để tối ưu Dijkstra xuống $O(E \log V)$; mở rộng kỹ thuật ghép cặp xử lý Euler sang đồ thị có hướng; xây dựng giao diện đồ hoạ cho phép nhập/chỉnh sửa đồ thị trực quan bằng chuột.

12. Hướng dẫn chạy chương trình

Yêu cầu môi trường

Python 3.8 trở lên

Thư viện: matplotlib, networkx (chỉ dùng cho vẽ hình) → `pip install matplotlib networkx`

Chạy demo tổng hợp toàn bộ 8 phần

```
012012204402_Group_12/src  
python3 main.py
```

Chương trình in kết quả từng phần ra màn hình và lưu toàn bộ hình ảnh minh hoạ vào graph project/outputs/.

Chạy riêng ví dụ xe quét rác

```
012012204402_Group_12/src  
python3 real_world.py
```

Thử với đồ thị khác

```
from graph import Graph
from route_orchestrator import route_orchestrator
g = Graph.from_file("../samples/street_network_cpp.txt")
result = route_orchestrator(g, start=0, verbose=True)
print(result["circuit"], result["total_distance"])
```

13. Tài liệu tham khảo

- Kenneth H. Rosen, Discrete Mathematics and Its Applications, McGraw-Hill.
- Thomas H. Cormen et al., Introduction to Algorithms (CLRS), MIT Press – các chương Graph Algorithms, Shortest Paths, Maximum Flow, Minimum Spanning Trees.
- Edmonds, J. & Johnson, E. L. (1973), Matching, Euler tours and the Chinese postman, Mathematical Programming.
- Bài giảng môn Cấu trúc rời rạc.
- Tài liệu thư viện matplotlib: <https://matplotlib.org/>
- Tài liệu thư viện networkx (chỉ dùng phần layout để vẽ hình): <https://networkx.org/>